FULL 7

Gestion de collection retro gaming

PLAN.

1 Contexte
2 Objectifs
3 Public cible
4 Utilisateurs
5 Choix techniques
6 Architecture
7 Conception de la base de données
8 Réalisation technique
9 Sécurité
10 Protection des données personnelles
11 Conclusion

1 Contexte

Les collectionneurs de jeux rétro peinent à suivre leur collection et à identifier les jeux qu’il leur manque.

1 Contexte

Les collectionneurs de jeux rétro peinent à suivre leur collection et à identifier les jeux qu'il leur manque.

La solution?

Full7, une application simple pour gérer sa collection et suivre sa progression.

2 Objectifs

Créer un compte et se connecter en toute sécurité.

Explorer un catalogue de consoles et de jeux.

Gérer sa collection personnelle.

Suivre sa progression grâce aux statistiques.

Personnaliser son profil et son interface.

3 Public cible

- **Joueurs et collectionneurs de jeux rétro.**

- **Accessible sur ordinateur et mobile**

4 Utilisateurs

Visiteur : connexion et inscription.

Utilisateur connecté : catalogue, gestion de collection, statistiques et personnalisation du profil.

aucun rôle administrateur dans cette version.

5 Choix techniques

Back-end : PHP pour gérer la logique et les sessions.

Base de données : MySQL / MariaDB pour stocker les utilisateurs, les jeux et les consoles.

Accès aux données : PDO et requêtes préparées pour sécuriser les échanges avec la base.

Front-end : HTML, CSS et JavaScript pour l'interface.

Environnement local : XAMPP pour exécuter le projet en local.

Versionnement : Git pour suivre les modifications du code.

6.1 Architecture

Organisation en dossiers : pages, styles, scripts et images séparés.

Factorisation : header, footer et connexion à la base centralisés.

Sécurité : identifiants de la base séparés du code principal.

Point d'entrée unique : redirection automatique selon la connexion de l'utilisateur.

6.2 Architecture

Ouverture technique : le modèle MVC

Modèle : gestion des données.

Vue : interface utilisateur.

Contrôleur : logique et traitement des requêtes.

Full7 utilise une architecture PHP organisée, mais ne suit pas strictement le modèle MVC.

7.1 Conception de la base de données

Une base de données en 4 tables:

CONSOLES : informations sur les consoles.

GAMES : catalogue des jeux.

USERS : comptes utilisateurs.

OWNERSHIPS : jeux possédés par chaque utilisateur.

Jeu de données : 10 consoles et 100 jeux intégrés au catalogue

7.2 Conception de la base de données

Organisation des relations:

Une console peut contenir plusieurs jeux.

Un jeu appartient à une console.

Un utilisateur peut posséder plusieurs jeux.

Un jeu peut appartenir à plusieurs utilisateurs.

Solution : la table ownerships relie les utilisateurs aux jeux et gère la relation plusieurs-à-plusieurs.

7.3 Conception de la base de données

Garantir la cohérence des données:

Clés primaires et étrangères : identifier et relier les données.

Contrainte UNIQUE : empêcher les doublons dans une collection.

ON DELETE CASCADE : supprimer les associations devenues inutiles.

Mots de passe hachés : éviter leur stockage en clair.

Script SQL de déploiement : recréer la base et importer les données.

8.1 Réalisation technique

Authentification et sessions

Inscription et connexion sécurisées.

Mots de passe hachés avec password_hash().

Vérification avec password_verify().

Gestion des utilisateurs connectés via les sessions PHP.

Requêtes préparées avec PDO pour limiter les injections SQL.

8.2 Réalisation technique

Calcul automatique de la progression

Nombre de jeux possédés.

Nombre de consoles collectionnées.

Pourcentage de progression globale.

Calculs réalisés à partir des données de la base.

Point technique : utilisation de jointures, de regroupements et de sous-requêtes SQL.

8.2 Réalisation technique

Ajouter, retirer et compléter sa collection:

Consultation des jeux par console.

Ajout et retrait des jeux possédés.

Mise à jour automatique des statistiques.

Badge Full 7 lorsque tous les jeux d'une console sont possédés.

Point technique : jointures SQL et `LEFT JOIN` pour compter les jeux possédés et le catalogue complet.

8.2 Réalisation technique

Personnalisation du profil:

Modification du pseudo et de la photo de profil.

Vérification du format des images envoyées.

Renommage des fichiers pour éviter les collisions.

Conservation de l'image par défaut si aucune photo n'est ajoutée.

9 Sécurité

Mots de passe : hachés avec password_hash() et password_verify()

Injection SQL : requêtes préparées PDO

Faille XSS : données échappées avec htmlspecialchars()

Accès non autorisé : vérification de session, redirection vers la connexion

Fichier malveillant : liste blanche d'extensions, getimagesize(), renommage

Identifiants BDD : secrets.php hors du dépôt Git

10 protection des données personnelles

Données minimales : pseudo, email, mot de passe, photo (facultative), jeux possédés

Mot de passe : jamais stocké en clair

Email : jamais affiché aux autres

Collection : visible uniquement par son propriétaire

Aucun suivi publicitaire, seul cookie : la session PHP

Limite actuelle : pas de suppression de compte par l'utilisateur (évolution prévue)

11 Conclusion

Objectif atteint :

11 Conclusion

Objectif atteint :gérer sa collection et suivre sa progression

11 Conclusion

Objectif atteint :gérer sa collection et suivre sa progression

Compétences travaillées :

11 Conclusion

Objectif atteint :gérer sa collection et suivre sa progression

Compétences travaillées : base de données, sécurité, PHP

11 Conclusion

Objectif atteint :gérer sa collection et suivre sa progression

Compétences travaillées : base de données, sécurité, PHP

Évolutions :

11 Conclusion

Objectif atteint :gérer sa collection et suivre sa progression

Compétences travaillées : base de données, sécurité, PHP

Évolutions : MVC, compte administrateur, suppression de compte, catalogue élargi

Merci